NAVION ROBOTICS · PRACTICAL USER GUIDE

Robot setup & user manual.

From opening the kit to a controlled first run.

One ESP32 chassis · Windows 10/11 · USB / Bluetooth remote · Line following · Obstacle avoidance

Revision 3 · 2026-11-05. Hardware assumptions require teacher verification.

Contents

1. Start here
2. Check the parts
3. Assemble the chassis
4. Install Arduino on Windows
5. Upload your first USB test
6. Wire the base robot
7. Check motor direction & stop
8. Read & calibrate the IR sensors
9. Drive from your Windows laptop
10. Set up the line follower
11. Set up obstacle avoidance
12. Shut down & change modes
13. Find & fix a problem
14. Understand the parts & code

CHAPTER 01

Start here

Build one robot, then set it up in three different ways.

BEFORE YOU START

A Windows 10/11 laptop, your robot kit and a teacher for hardware checks.

FINISH WHEN

You know the order of work and which instructions to follow.

Your first goal

Make the robot move forward briefly and stop when you tell it to. After that, reuse the same chassis for line following and obstacle avoidance. This guide tells you what to connect, what to click, what to upload and what result to look for.

Check the kit | Build the chassis | Install Arduino | Test USB | Wire & test motors | Choose a robot mode

Follow this order

Stage	Do this	Finish when…
1 · Get ready	Check the parts → assemble with power off	The wheels turn freely and boards are secure.
2 · Talk to the ESP32	Install Arduino IDE → run the USB test	PING produces PONG in Serial Monitor.
3 · Make movement predictable	Connect the provisional circuit → check motor direction	Both wheels move forward and the stop test works.
4 · Add sensing	Read and calibrate the IR sensors	You recorded the actual HIGH/LOW meaning.
5 · Select a mode	Laptop remote / line follower / obstacle avoidance	The correct program is uploaded and a supervised test is logged.

How to use each chapter

Read **Before you start**, follow one numbered step at a time, and compare your result with **You should see**. Use the "If that does not happen" advice before moving on. The checkboxes are a paper-style checklist; they are not saved and do not unlock anything.

You can read every page and download every sketch immediately. No account, payment or completion score is needed.

> **Hardware checks still need your teacher**
>
> This is a software-checked guide, not a physically validated kit. The exact ESP32 pin labels, IR module voltage/polarity, L298N regulator output, motor fit and USB/external-power isolation must be checked on the supplied kit. Do not interpret a diagram or compiled sketch as approval to power an unverified circuit.

Choose a mode after the base setup

Mode	Sensor position	How it works
Laptop remote control	Sensors not required	Windows USB Serial first; Bluetooth Classic COM where supported. Send F/B/L/R/S.
Line follower	Left and right modules face down, either side of the line	Black line on white, about 25.4 mm wide. The program steers from two digital readings.
Obstacle avoidance	Modules face left, front and right	Tune for roughly 5–10 cm on the real object. Prefer left if both sides are clear.

When to stop and ask

Stop if you cannot identify a power terminal, a wire is loose, a wheel is jammed, anything heats up, cells are damaged, or the ESP32 repeatedly restarts. Switch off the motor battery before inspecting. Charging and cell preparation are adult tasks.

CHAPTER 02

Check the parts

Find each part and learn its job before connecting anything.

BEFORE YOU START

The kit laid out on a clear table. Keep cells with the teacher.

FINISH WHEN

Every item is accounted for, and missing or damaged parts are reported.

Inventory for one reusable kit

Use this exact list. The third IR module is spare during line following, then used in obstacle mode.

Part / specification	Per kit
ESP32-WROOM-32-family 30-pin development board, CP2102, USB Type-C, fitted headers, Classic + BLE	1
Red L298N 2 A module, heatsink and fitted terminals	1
Adjustable 3-pin digital IR modules, intended 3.3 V operation	3
Matched 18650 Li-ion cells, 3.7 V nominal, 2000 mAh	2
Two-cell series holder with switch and attached leads	1
Prewired 130–280 RPM BO geared motor, user-rated 3–9 V	2
65 mm yellow drive wheels matching shafts	2
Motor clamps matching chassis	2
Small caster	1
SYB-170 mini breadboard	1
20 cm male–female jumper wires, 20 per pack	1 pack

20 cm female–female jumper wires, 20 per pack	1 pack
USB-A to USB-C data cable, 0.5 m	1
Small 2-in-1 flat + Phillips screwdriver	1

Navion supplies the separate 3D-printed chassis and interchangeable sensor mounts. Fastener dimensions and quantities are not confirmed yet; the teacher must match the actual hardware to the chassis.

Identify the important labels

Part	Look for	Its job
ESP32 development board	USB Type-C connector; printed pin labels; WROOM module	Runs your program. Connect only to confirmed board pins.
L298N driver	OUT1–OUT4; motor supply, GND, 5 V; IN1–IN4; ENA/ENB	Drives the motors. A GPIO does not power a motor directly.
IR module	VCC, GND and OUT labels on the actual board	Reports a digital state based on reflected infrared light. Pin order varies.
Switched series holder	Switch plus pre-attached leads	Holds two cells in series. A switch is not evidence of battery protection.
Data cable	USB-A plug for laptop, USB-C for this board	Provides USB communication as well as power.

STEP 1

Count, inspect and sort

Count parts against the table. Put motor parts together, electronics together, and jumpers together. Leave the holder switch OFF. Ask the teacher to inspect cells, fitted wires, headers and terminals.

> **You should see**
>
> No damaged insulation, bent headers, missing parts or loose motor wires.

If that does not happen: Report a missing item rather than substituting an unknown part.

STEP 2

Check the cable before blaming the board

Use the supplied data-capable cable. A charge-only cable can power a board but will not create a COM port.

> **You should see**
>
> The cable is labelled or teacher-confirmed for data. You will verify it in the USB test.

If that does not happen: Try a known working data cable during software setup.

Ready to continue

- [ ] All quantities checked.
- [ ] Teacher has the cells and has identified the holder polarity.
- [ ] The actual motor/driver/sensor labels can be read.
- [ ] Missing parts or unconfirmed fasteners are reported.

> **About pictures**
>
> Genuine kit and assembly photographs have not been supplied yet. Use the printed component labels and teacher verification; do not infer a physical pin order from a generic photograph.

CHAPTER 03

Assemble the chassis

Build the base with power off; leave the sensor arrangement easy to change.

BEFORE YOU START

Chassis, motors, clamps, wheels, caster, boards, holder, screwdriver and teacher-approved fasteners.

FINISH WHEN

A stable chassis with clear wheel movement and accessible USB and power switch.

Keep all power disconnected

Holder switch OFF, cells handled by the teacher, and USB unplugged. Do not use an unknown screw length: it may damage a printed mount or the motor casing.

Build in this order

STEP 1

Lay out the chassis

Identify front and rear with your teacher. Check the mounting holes for both motor clamps and the caster. Choose a flat surface; keep small fasteners in a tray.

You should see

Both motors can sit on opposite sides, with their shafts aligned across the chassis.

If that does not happen: Exact chassis hole positions and screw sizes remain TBD; ask the teacher to match the supplied printed parts.

STEP 2

Fix both motors in their clamps

Attach each clamp to its intended mount, then secure the motor body evenly. Keep motor wires facing a route that avoids shafts and wheels. Tighten gently and alternately.

You should see

The motor cannot shift inside its clamp and its gearbox is not squeezed or cracked.

If that does not happen: Loosen and realign. Do not solve rubbing by forcing the wheel.

STEP 3

Fit the two wheels and caster

Match the wheel hub to the motor shaft. Press straight while supporting the motor; do not hammer. Fit the caster in its intended mount. Rotate each wheel by hand.

> **You should see**
>
> Both wheels and the caster touch a level surface; wheels rotate without touching the chassis or wires.

If that does not happen: Check hub fit, clamp alignment and caster height with the teacher.

STEP 4

Mount electronics and holder

Use the intended supports and teacher-approved fasteners. Leave an insulating gap under circuit boards. Keep the L298N heatsink clear. Secure the holder so it cannot slide. Keep the switch and USB socket reachable.

> **You should see**
>
> Boards and holder stay in place when the chassis is gently lifted. No terminals touch metal fasteners.

If that does not happen: Reposition before wiring. Do not leave loose boards on top of a battery holder.

STEP 5

Prepare the sensor mounts

Use two downward mounts for line mode, or three directional mounts for obstacle mode. For your first motor test, sensors are not needed. Route sensor cables so they can be moved later without pulling a connector.

> **You should see**
>
> No sensor or wire reaches the ground or catches a wheel.

If that does not happen: The correct downward height and directional angles must be calibrated on the actual chassis; no fixed geometry has been validated.

Mechanical check before software setup

- [ ] Wheel rotation is free on both sides.
- [ ] No wire can reach a wheel or caster.
- [ ] Boards are secured and electrically isolated from fasteners.
- [ ] Holder switch and USB connector are reachable.
- [ ] Teacher has checked fasteners; power remains disconnected.

CHAPTER 04

Install Arduino on Windows

Prepare the IDE and find the ESP32 USB port.

BEFORE YOU START

Windows 10/11 laptop, internet for installation, ESP32 and data cable. Battery OFF; external ESP32 5 V lead disconnected.

FINISH WHEN

Arduino IDE recognizes a USB COM port and the original ESP32 board target.

What you are installing

Arduino IDE is the editor and upload tool. The Espressif board package teaches it how to compile for ESP32. A CP210x driver may be needed for Windows to recognize this board's USB-to-serial chip. Downloads are from the official suppliers.

Install and select the board

STEP 1

Install Arduino IDE 2.x

Open Arduino's software page. Choose the Windows installer for IDE 2.x. Follow the installer, then open Arduino IDE. Ask lab staff if administrator permission is needed.

> **You should see**
>
> A new sketch window containing setup() and loop().

If that does not happen: Ask the teacher to complete installation on managed lab computers.

STEP 2

Add the Espressif package URL

Open **File** → **Preferences**. Find **Additional Boards Manager URLs**. Paste this URL on its own line, preserving any existing entries, and press OK.

```
https://espressif.github.io/arduino-esp32/package_esp32_index.json
```

> **You should see**
>
> The full URL is saved in Preferences.

If that does not happen: Check there are no spaces inside the URL or missing characters.

STEP 3

Install esp32 by Espressif Systems

Open **Tools** → **Board** → **Boards Manager** or the Boards Manager icon. Search `esp32`. Choose **esp32 by Espressif Systems**, select a **3.x** version, and Install. Wait for completion. The robot sketches were compiled with version **3.3.11**.

> **You should see**
>
> Installation finishes without an error. ESP32 boards appear in the board selector.

If that does not happen: Check network access and disk space. Core 2.x uses different PWM APIs; keep this guide and core 3.x together.

STEP 4

Connect the board by USB only

Keep the motor battery OFF and have the teacher disconnect the external 5 V feed at ESP32. Plug the data cable directly into the laptop and ESP32. In Windows, open **Device Manager → Ports (COM & LPT)**. Unplug/replug once if needed to identify the changing entry.

> **You should see**
>
> A USB serial entry, often labelled Silicon Labs CP210x, with a COM number such as COM5. Your number may differ.

If that does not happen: If only power appears, try a known data cable/USB port. If Device Manager reports a CP210x driver problem, use the official Silicon Labs driver with teacher approval.

STEP 5

Select board and port in the IDE

Choose **Tools → Board → esp32 → ESP32 Dev Module** for the supplied original WROOM-32 target. Choose the observed USB COM port under **Tools → Port**. Confirm the physical board with the teacher; this is not an ESP32-C3/S3 target.

> **You should see**
>
> Board selector shows ESP32 Dev Module and your actual COM port.

If that does not happen: Do not guess a port from another student's computer. Close applications that already use yours.

Before the first upload

- [ ] Arduino IDE 2.x opens.
- [ ] Espressif core 3.x is installed.
- [ ] ESP32 Dev Module and the observed USB COM port are selected.
- [ ] External 5 V is isolated while USB powers the board.

What the buttons do

The tick/check button **Verify** compiles the sketch. The arrow **Upload** compiles and sends it to the board. **Serial Monitor** displays messages; it is not the code editor. A successful compile alone does not mean the board has been programmed.

CHAPTER 05

Upload your first USB test

Prove the laptop can talk to the ESP32 before motors are connected.

BEFORE YOU START

IDE installed, board/port selected, USB only, motor battery OFF and external 5 V isolated.

FINISH WHEN

Sending PING returns PONG at 115200 baud.

Download a ready-to-open test

You do not need to type the code by hand. Use the download, place it in the correct folder, and open that file.

Download USB test (.ino)

Upload and check

STEP 1

Put the sketch in its own folder

In File Explorer, make a folder named `00_Serial_Test`. Move the downloaded `00_Serial_Test.ino` into it. In Arduino IDE choose **File** → **Open** and open that file. If the IDE offers to create/move it into a matching folder, accept.

You should see

An IDE tab named 00_Serial_Test containing setup() and loop().

If that does not happen: Windows may hide extensions. Enable View → Show → File name extensions to avoid a file ending in .ino.txt.

STEP 2

Verify, then Upload

Check **ESP32 Dev Module** and the USB COM port. Click the tick/check **Verify** button. When compilation finishes, click the arrow **Upload** button. Wait for transfer and reset to finish.

> **You should see**
>
> Verify has no compiler error, then Upload finishes without a connection error.

If that does not happen: A progress bar alone is not success. Read the final output; see the troubleshooting chapter for compile, port-busy or connection errors.

STEP 3

Open Serial Monitor

Open **Tools → Serial Monitor**. Select **115200 baud** and line ending **Newline**. Press the board's EN/RESET button once if its startup message was missed. Do not hold BOOT for this reset.

> **You should see**
>
> `Ready. Send PING with Newline.`

If that does not happen: Close another terminal using the port. Check 115200 baud if text is unreadable, then confirm the correct sketch was uploaded.

STEP 4

Send PING

Click the Serial Monitor's message/input box, type `PING` in capitals, and press Enter/Send. Type into Serial Monitor, not into the C++ editor.

> **You should see**
>
> `PONG` appears as the board's response. Repeat once to confirm.

If that does not happen: If the board says "Type PING in capitals", check spelling. If nothing arrives, check Newline, selected port and Upload result. The Serial Plotter is a different tool.

Do not move on until this works

- [ ] Correct sketch uploaded without error.
- [ ] Serial Monitor is 115200 / Newline.
- [ ] PING gives PONG.
- [ ] COM number recorded for later: _____.

> **About the program**
>
> setup() runs once and starts serial. loop() checks incoming text. The teaching sketch waits briefly for a newline; the remote-control sketch uses a different nonblocking parser so it can check its stop timeout.

CHAPTER 06

Wire the base robot

Connect the driver and power path only after the teacher checks the actual modules.

BEFORE YOU START

USB test passed; battery and USB disconnected; teacher present; readable pin labels.

FINISH WHEN

Every base connection is checked against a table before power is applied.

Read the power procedure first

Teacher check · provisional wiring

The assumed power path is 2 × matched 18650 cells in series (7.4 V nominal, 8.4 V full) → switched holder → L298N motor supply → onboard regulated 5 V → ESP32 designated 5V/VIN. It is not electrically verified. The regulator may drop out as batteries discharge. Never connect this supply to 3V3.

Keep the regulator 5V-EN jumper as required by the actual module for its 5 V output. Remove only ENA/ENB motor-enable jumpers for PWM. These are different jumpers. Teacher must inspect the exact board before connecting USB and external 5 V together. Default upload procedure: battery off, disconnect the external 5 V wire from ESP32, use USB, then unplug USB before restoring external power.

Use common ground. Sensors must work at 3.3 V with driven digital outputs; no 5 V on GPIO. GPIO 34/35/39 are input only and have no internal pull-ups. An adult prepares matched cells; avoid shorts and improper charging. Switch off during assembly and use teacher supervision. A switched holder does not establish battery protection.

1 · Choose the correct power state

State	USB	Motor battery	External 5 V lead at ESP32
Uploading / USB diagnostic	Connected	OFF	Disconnected / isolated by teacher
Battery-powered floor run	Unplugged	Teacher switches ON after checks	Connected only after regulator validation
Changing wires / mounts	Unplugged	OFF	No changes while energized

USB motor testing needs a teacher-approved arrangement

For a motor test while USB remains connected, the teacher must isolate the driver's external 5 V feed from the ESP32 and confirm the exact development-board circuit. USB powers the ESP32; the separate motor supply powers the L298N; grounds stay common. If this arrangement is not approved, do not turn on the battery during USB use. The alternative is a teacher-approved battery-only Bluetooth test after pairing.

2 · Connect ground, then motor outputs

From	To	Check
ESP32 GND	L298N GND	Common reference for control signals
Left motor pair	L298N OUT1 and OUT2	Never connect motor leads directly to GPIO
Right motor pair	L298N OUT3 and OUT4	Keep both leads in the intended output pair
Holder negative (teacher identifies)	L298N GND supply terminal	Do not rely only on wire colour
Holder positive through its switch	L298N motor supply terminal marked +12V / VS on actual module	This terminal label does not change the 2S assumption

3 · Connect ESP32 control signals

Match GPIO numbers, not physical “pin 25” positions. Read the board silkscreen. The map is provisional until the supplied 30-pin board is checked.

Function	GPIO	Note
IN1	25	L298N direction
IN2	26	L298N direction
IN3	27	L298N direction
IN4	14	L298N direction
ENA	32	Motor PWM, remove enable jumper
ENB	33	Motor PWM, remove enable jumper
LEFT_IR	34	Driven 3.3 V digital input
FRONT_IR	35	Driven 3.3 V digital input
RIGHT_IR	39	Driven 3.3 V digital input

ENA/ENB: remove the two motor enable jumpers before connecting GPIO32/33 for PWM. **5V-EN:** the regulator jumper is separate; its setting must match the teacher-approved onboard-regulator arrangement. Never remove all jumpers as one step.

4 · Connect the assumed ESP32 power feed

Only after a teacher validates the module's regulated output, connect L298N **5 V output** to the development board's designated **5V/VIN input**. Do not use 3V3. Connect or disconnect this lead only with battery OFF and USB unplugged. Some modules lose regulation as a 2S battery falls; stability under motor load is unverified.

5 · Add sensors later

You can leave IR modules disconnected for the first motor test. For sensor work, VCC goes to confirmed 3V3, GND to common GND, OUT to the assigned input. Pin order varies by module. GPIO34/35/39 have no internal pull-ups and require a suitable driven digital output. A 5 V output must never be applied directly.

Provisional connection overview

NAVION · PROVISIONAL / NOT HARDWARE VERIFIED

- 2S holder → L298N VS
- L298N 5 V → ESP32 VIN (ASSUMED)
- Common GND · sensors 3V3 only
- 25/26 left IN · 27/14 right IN
- 32 ENA · 33 ENB (remove motor jumpers)
- IR: left 34 · front 35 · right 39

Teacher check before any battery test

- [ ] Exact ESP32 board pins and VIN label confirmed.
- [ ] No shorts, reversed supply or loose terminal strands.
- [ ] Left/right motor pairs and IN1–IN4 checked.
- [ ] ENA/ENB PWM jumpers removed; regulator jumper checked separately.
- [ ] Assumed regulator output and USB/external feed isolation approved.
- [ ] Switch reachable; wheels raised; test area clear.

CHAPTER 07

Check motor direction & stop

Use short USB commands before letting the robot touch the floor.

BEFORE YOU START

Base wiring approved; wheels lifted; remote sketch downloaded; teacher-approved test power arrangement.

FINISH WHEN

F turns both wheels forward; S and command timeout stop them.

> **Power choice comes before the motor test**
>
> Uploading uses USB only, battery OFF and external ESP32 5 V feed isolated. To run motors with USB connected, use only the teacher-approved isolation arrangement described in Power & wiring. If that is unavailable, do not energize the motor supply during USB use; finish Bluetooth pairing with motors OFF, then return here for the lifted-wheel tests using the approved battery-only arrangement. If pairing fails too, stop for the teacher; neither power path is approved.

Upload the remote sketch

Download motor / remote sketch

STEP 1

Open and upload 01_Remote_Control

Put `01_Remote_Control.ino` inside a folder named `01_Remote_Control`. Open it, select the original ESP32 board and USB COM port, Verify, then Upload. Keep the motor supply OFF.

> **You should see**
>
> At 115200 baud, the program prints `Ready F B L R S; repeat movement within 800 ms`.

If that does not happen: Check the selected core is 3.x and use the matching .ino, not the serial PING program.

STEP 2

Send a stop before powering the motors

In Serial Monitor send uppercase `S`. After your teacher approves the power arrangement, keep wheels lifted and turn on the motor supply. Keep the switch within reach.

> **You should see**
>
> The board prints `STOP`; wheels remain stopped until a movement command.

If that does not happen: Switch the battery OFF if a wheel moves unexpectedly; inspect driver enable wiring and the uploaded program.

STEP 3

Check one brief forward command

Send `F` once. Observe both wheels. “Forward” means the wheel rotation would move the chassis toward its marked front. The shafts face opposite sides, so do not judge by the same clockwise direction from both outside views.

> **You should see**
>
> Both wheels run briefly, then stop in about 800 ms without another movement command.

If that does not happen: If the board prints F but a wheel is still, stop and check ENA/ENB, motor output pair, supply and friction. Do not increase speed against a jam.

STEP 4

Fix a reversed motor in code

Switch the battery OFF and isolate external power before uploading again. At the top of the sketch, change only the affected constant from false to true:

```
const bool INVERT_LEFT=true, INVERT_RIGHT=false; // example: only left needed reversing
```

Upload and repeat the short F test. If both are reversed, change both. Later copy these same inversion settings into the line and obstacle sketches.

> **You should see**
>
> F now drives both wheels toward the robot's front.

If that does not happen: Check you edited the correct sketch and uploaded after saving. Do not swap live motor wires.

STEP 5

Check turning and stop behavior

Try `B`, then `L`, then `R` one at a time. Send `S` after each. For the timeout check, send one F and send nothing afterward.

> **You should see**
>
> B reverses; L turns left; R turns right. S requests an immediate stop when processed, and silence after F produces a timeout stop.

If that does not happen: Keep wheels raised and switch OFF if the direction/stop behavior is unexpected. Fix that before floor testing.

Ready for a floor run?

- [ ] Both forward directions are correct.
- [ ] Stop command and 800 ms timeout checked.
- [ ] Inversion settings recorded: LEFT _____ RIGHT _____.
- [ ] Wires clear, no rubbing or repeated reset.
- [ ] Teacher approves a short, low-speed run on a clear floor.

CHAPTER 08

Read & calibrate the IR sensors

Measure what LOW and HIGH actually mean on your kit.

BEFORE YOU START

Teacher-confirmed 3.3 V modules, USB diagnostic power only, motor battery OFF, black/white material and a test obstacle.

FINISH WHEN

A recorded raw state for each surface/object and a matching DETECTED_LEVEL setting.

Use a diagnostic that cannot move the robot

Leave the motor battery OFF. Keep the external ESP32 5 V feed disconnected while using USB. The sensor diagnostic does not initialize the motor driver; it only reads inputs.

Connect only verified 3.3 V modules

Module connection	ESP32 connection
VCC on each confirmed-compatible module	3V3
GND	GND / common ground
Left OUT	GPIO34
Front OUT	GPIO35
Right OUT	GPIO39 / VN, if exposed

Do not assume all three-pin sensors have the same pin order. Read the labels on each board. Do not use INPUT_PULLUP on 34/35/39; these inputs have no internal pull-ups.

Read the raw states

Download sensor diagnostic

STEP 1

Upload the sensor diagnostic

Place `00_Sensor_Check.ino` in a same-named folder. Upload over USB. Open Serial Monitor at 115200 baud. Keep motor battery OFF.

> **You should see**
>
> Rows such as `Left=1 Front=1 Right=1` about five times per second. These are example values, not expected universal readings.

If that does not happen: Check OUT/VCC/GND mapping. A disconnected input may float and change randomly; ignore readings for any module not connected.

STEP 2

Calibrate for line following

Place the left module at its intended downward height. Move the same black tape and white floor under it. Record both raw values. Adjust the sensitivity potentiometer gently until the states differ reliably. Repeat with the right module at the same height and lighting.

> **You should see**
>
> Each module changes between LOW (0) and HIGH (1) when black/white changes. You can say exactly which state means black.

If that does not happen: If states never change, check voltage compatibility, pin labels, mounting height, sensitivity and ambient light. Indicator LEDs may indicate power or detection depending on the module; use the raw output as evidence.

STEP 3

Calibrate for obstacles separately

Turn off/unplug before moving mounts. Face modules left/front/right. Under USB diagnostic power, move a representative matte object toward each module. Tune roughly 5–10 cm on that actual object. Record state with and without it.

> **You should see**
>
> A repeatable state change for the chosen object at the chosen distance.

If that does not happen: IR depends on reflectivity and light. Dark, transparent or shiny objects may be missed. The distance target is not a guarantee.

STEP 4

Set the program's meaning

In the relevant robot sketch use `DETECTED_LEVEL=LOW` if 0 means black (line mode) or obstacle (avoidance mode). Otherwise use `HIGH`. The supplied sketches use one shared active-level constant, so all connected modules must agree after calibration.

> **You should see**
>
> The program's detected() meaning matches your measured inputs.

If that does not happen: If modules have opposite detection polarity, ask the teacher to adapt per-sensor logic; do not assume one global inversion fixes a mismatched pair.

Write down the result

Sensor	White / clear raw state	Black / obstacle raw state	Height / distance / light
Left	____	____	____
Front (obstacle only)	____	____	____
Right	____	____	____

After calibration, upload the chosen robot sketch again. The diagnostic does not run a robot mode.

CHAPTER 09

Drive from your Windows laptop

Start with short USB commands; then try a Bluetooth Classic COM port.

BEFORE YOU START

Motor direction and stop tests passed; remote sketch loaded; teacher-approved power setup.

FINISH WHEN

You can drive in short movements and reliably stop through the selected serial connection.

1 · Start with USB control

If you have just completed the motor test, the correct sketch is already loaded.
Otherwise download it here and follow the first USB upload procedure.

Download remote sketch

Send	Movement	Expected feedback
F	Forward	F, then Timeout stop if no repeat
B	Backward	B
L	Turn left	L
R	Turn right	R
S	Stop	STOP

Use uppercase single letters, one command at a time. The parser ignores line-ending characters. Do not type “forward” or PING into this sketch. PING works only in the separate USB diagnostic.

Why one press only moves briefly

For safety, a valid movement command must arrive again within **800 ms**. The IDE's Send box is useful for short test movements, not smooth continuous steering. A serial terminal that sends each key immediately can be more convenient. Initial operating-system key repeat can be slower than the timeout; test it with wheels raised. Do not remove the failsafe to make driving easier.

STEP 1

Make a short supervised floor test

Only after the lifted-wheel tests pass, place the robot in a clear area. Send F once and observe, then S. Test B/L/R briefly. Keep the power switch reachable and stop if the USB cable could catch a wheel.

You should see

Short, predictable movements that agree with the command table.

If that does not happen: If it veers, check wheel friction and motor orientation before changing PWM. If it resets, stop for a teacher power check.

2 · Pair Bluetooth on Windows

The original ESP32 supports Bluetooth Classic SPP. This guide assumes your Windows adapter exposes its serial service; that actual connection has not been tested. Pairing alone does not prove serial communication.

STEP 2

Find NAVION_ROBOT

With the remote sketch running and motor battery OFF, open **Windows Settings → Bluetooth & devices → Add device → Bluetooth**. Choose **NAVION_ROBOT**. In a classroom, the teacher should first change the BT.begin name in each kit to a unique label and re-upload.

You should see

The intended board is listed as paired.

If that does not happen: Check you have the original WROOM-32 with Classic SPP, not a C3/S3 BLE-only substitute. Restart the board and Bluetooth adapter if needed; avoid connecting to another kit.

STEP 3

Find its outgoing COM port

Look in **More Bluetooth settings** → **COM Ports** and in Device Manager. Find the outgoing COM port for that paired device's serial service. The Bluetooth number can differ from the USB COM number. Close Serial Monitor before another terminal opens a port.

> **You should see**
>
> A distinct outgoing serial COM entry for your robot.

If that does not happen: Some Windows configurations do not offer a working Classic serial port. Use USB as the fallback instead of installing a phone app or expecting browser Bluetooth to work.

STEP 4

Test the wireless serial connection

Try the outgoing COM in Arduino Serial Monitor if available, or open a Windows serial terminal such as PuTTY from its official site: choose Serial, enter the outgoing COM number and 115200 speed. Send S first. Start with motors OFF; after communication works, have the teacher arrange approved battery-only power with USB unplugged. Connect to the COM port again if needed.

> **You should see**
>
> The robot replies STOP to S over the wireless terminal. With approved battery-only power and wheels raised, one F causes a brief movement.

If that does not happen: Pairing, a terminal window, or an open COM port alone is not proof. Look for actual command feedback. Check correct outgoing port, one application using it, and powered board.

STEP 5

Test disconnect and timeout stops

With wheels raised, send F over Bluetooth and close/disconnect that terminal. Check it stops. Reconnect and repeat one F with no following command. Use the power switch if needed.

> **You should see**
>
> Bluetooth-owned motion stops on disconnect; the 800 ms command timeout also stops motion.

If that does not happen: A Bluetooth disconnect does not cancel a more recent USB movement command, because the last movement sender owns the motion. S from either source stops it. Use one controller at a time for student tests.

Settings you may tune

Constant	Starting setting	Meaning
SPEED	145	8-bit PWM duty 0–255, not measured RPM. Lower may fail to start the motors.
INVERT_LEFT / INVERT_RIGHT	Use your recorded motor-test values	Copy the correct wheel direction settings into every robot sketch.
DEADMAN_MS	800	Stop timeout; retain this failsafe.

Finished when

- [ ] F/B/L/R match physical directions.
- [ ] S and timeout work before floor use.
- [ ] If Bluetooth is used, feedback and disconnect stop were actually observed.
- [ ] USB fallback is understood if no wireless COM works.

CHAPTER 10

Set up the line follower

Reposition two sensors and upload the two-sensor program.

BEFORE YOU START

Base robot tested, IR polarity recorded, two downward mounts, white matte surface and about 25.4 mm black tape.

FINISH WHEN

A short supervised test demonstrates steering and a bounded stop.

Know this program's limitation first

Both white does not uniquely tell us where the robot is

With **sensor — line — sensor**, both white may mean the line is correctly between the sensors, or that the robot has left the track entirely. The delivered program uses a conservative limit: forward for up to 700 ms without a black detection, then search in the last turn direction for up to 1500 ms, then stop. A long straight can therefore trigger a search. Treat this as a starter demonstration, not guaranteed continuous track following.

Prepare and upload

Download line-follower sketch

STEP 1

Make a simple test track

Place a black line about **25.4 mm / 1 inch wide** on a white matte surface. Start with a short gentle bend in a clear area rather than a complicated maze, intersection or long straight. Avoid reflective tape and direct sunlight.

You should see

A smooth, high-contrast line with room to catch and stop the robot.

If that does not happen: Change the track surface if calibration cannot separate black and white reliably.

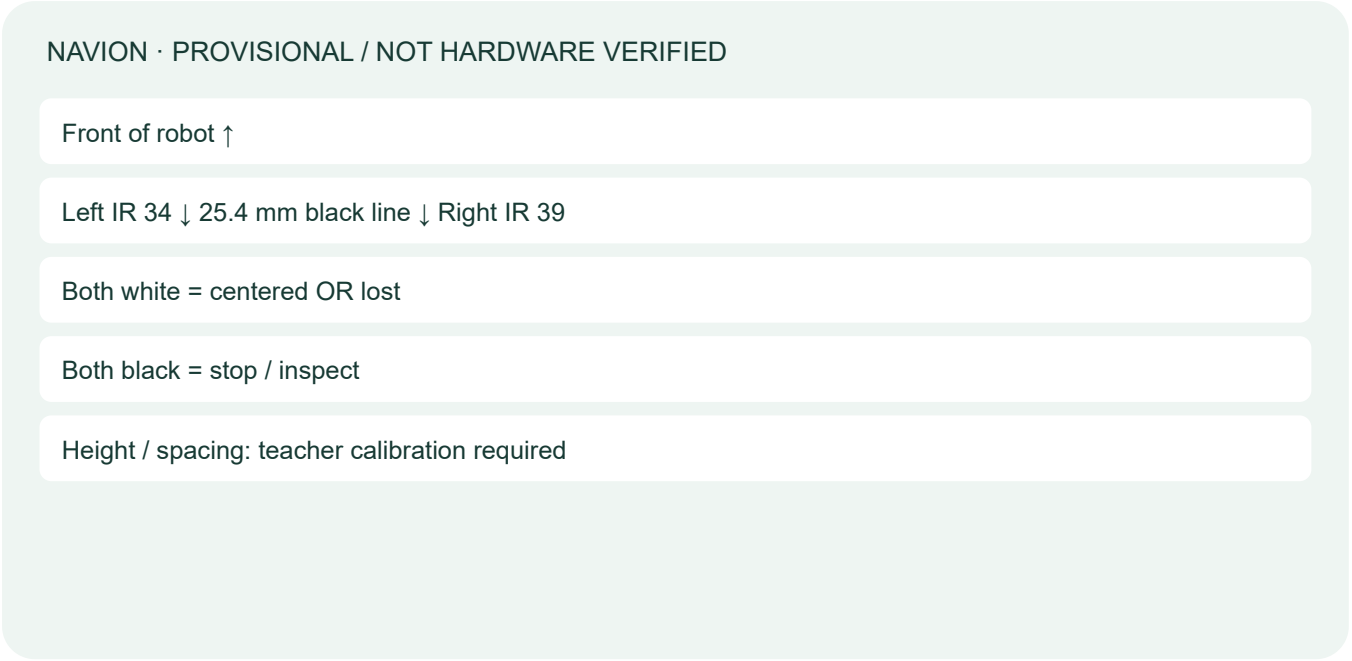
STEP 2

Mount and wire two downward sensors

With all power off, place left and right sensors near the front, facing down and flanking the line. Use the same height and spacing you calibrated. Connect left OUT to GPIO34 and right OUT to GPIO39; keep 3V3 and GND verified. The third/front sensor is not needed.

> **You should see**
>
> When centered, the line lies between the sensors and both nominally see white.

If that does not happen: Do not guess a fixed mounting height; recalibrate on the actual printed mounts.



STEP 3

Copy your calibration settings

Open the line sketch. Set `INVERT_LEFT` and `INVERT_RIGHT` to the motor-test values. Set `DETECTED_LEVEL` to the raw value that actually meant black for both sensors. Keep the initial SPEED at 145 until motion is understood.

> **You should see**
>
> The source settings agree with your motor and sensor notes.

If that does not happen: If only one sensor has the opposite polarity, ask for per-sensor code adaptation before running.

STEP 4

Upload with motor battery OFF

Isolate external 5 V and use USB to upload the .ino from its matching folder. Use ESP32 Dev Module / core 3.x. This program begins an autonomous 60-second session when the board starts. Do not turn on the motor supply just to inspect Serial Monitor.

> **You should see**
>
> Upload completes. Serial output identifies line mode.

If that does not happen: Verify the correct file, board and port; keep wheels raised whenever a motor supply is enabled.

Understand the steering table

Left sees black	Right sees black	This sketch does
No	No	Forward briefly, then bounded last-direction search, then latch stopped
Yes	No	Turn left
No	Yes	Turn right
Yes	Yes	Stop and latch for crossing/alignment inspection

STEP 5

Check the four inputs with wheels lifted

With your teacher controlling a safe power arrangement, present white/black under each sensor and compare the wheel action with the table. Test both black and then reset only after inspection to restart. For a search test, present white to both and observe the bounded stop.

> **You should see**
>
> Turning follows the table; both black latches stopped; an unrecovered search also stops.

If that does not happen: Switch off if response is reversed. Check polarity, OUT pins and motor inversion separately.

STEP 6

Run a short floor test

For approved battery-only running, unplug USB, restore the verified external feed with power OFF, place on the track and clear the area. Teacher switches ON; the program begins automatically. Keep a hand near the switch. Switch OFF before repositioning.

> **You should see**
>
> A short steering demonstration. Search can occur on straights because of the two-sensor ambiguity.

If that does not happen: If it loses the track, do not let it spin indefinitely. The code latches stopped; switch off, reposition and restart after checking spacing, speed and polarity.

Tune one value at a time

Setting	What to observe
SPEED	Lower values may make steering calmer but can stall a motor.
SEARCH_MS = 1500	Maximum search interval after the 700 ms forward window.
lastTurn = -1	Initial search direction is left; +1 means right.
SESSION_MS = 60000	Stops autonomous motion after one minute; restart only after inspection.

Finished when

- [ ] Mounting and raw polarity recorded for both sensors.
- [ ] Wheel direction settings copied correctly.
- [ ] All four combinations checked with wheels lifted.
- [ ] Both-black and bounded-search stop observed.
- [ ] One short track test logged, including any straight-line limitation.

CHAPTER 11

Set up obstacle avoidance

Move three sensors to the sides and front, then test each decision.

BEFORE YOU START

Base robot tested, three confirmed IR modules, safe matte obstacles, cleared rear space and teacher supervision.

FINISH WHEN

The robot chooses a clear side and stops its timed session.

What this mode does

It advances while the front is clear. When the front detects an obstacle, it stops and chooses an unblocked side. If both sides are clear, it prefers left. If all three detect obstacles, it reverses briefly and makes a timed left U-turn. No ultrasonic sensor, servo or Wi-Fi is needed.

> **No rear sensor is fitted**
>
> Reverse motion is timed and cannot detect something behind the robot. Keep rear space clear and guard the robot during tests. IR may miss dark, transparent or shiny objects; do not use a person or a drop-off as a test obstacle.

Reconfigure and upload

Download obstacle sketch

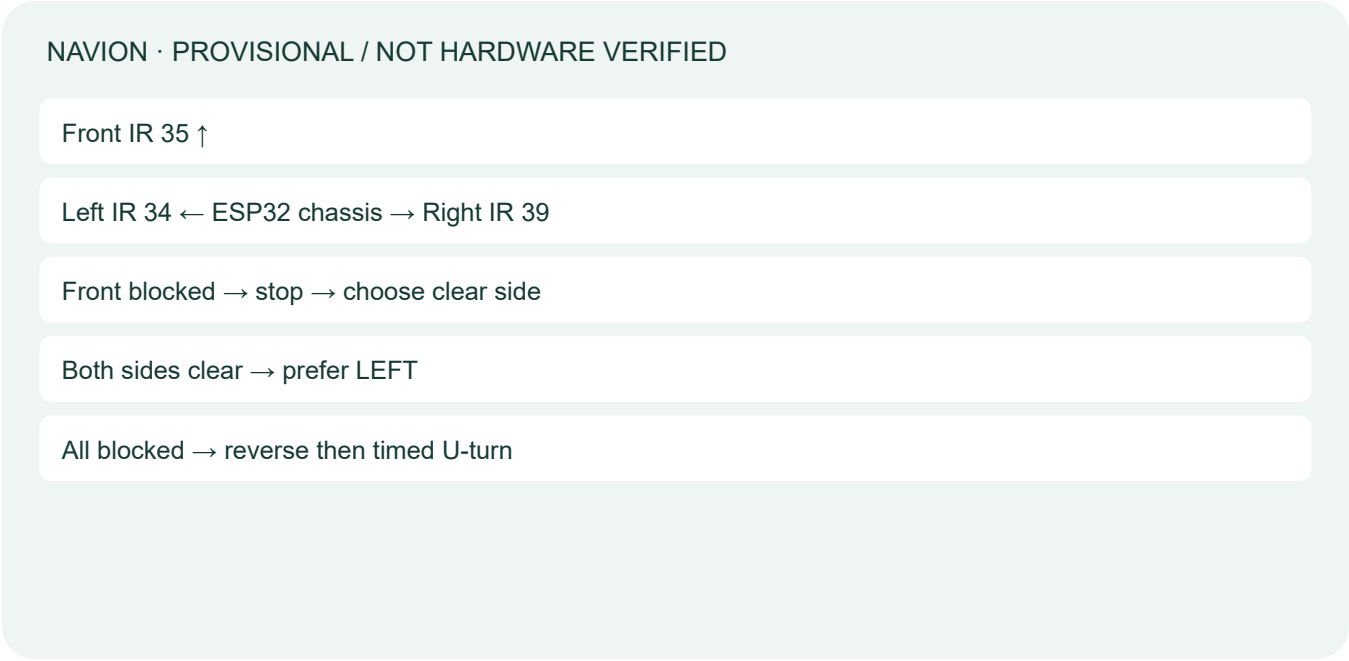
STEP 1

Fit three directional sensors

With power off and USB unplugged, move modules to face left, front and right. Connect OUT to GPIO34, GPIO35 and GPIO39 respectively. Connect each verified VCC to 3V3 and GND to common ground. Keep wiring away from wheels.

> **You should see**
>
> The front module looks ahead; side modules cover their intended directions without the chassis blocking them.

If that does not happen: Exact mounting angles depend on the printed mounts and are not yet physically validated.



STEP 2

Recalibrate for actual objects

Use the motor-OFF sensor diagnostic. Adjust each module for roughly 5–10 cm on the actual matte object and room lighting. Record clear/detected raw values for all three. A setting that worked on floor tape may not work on a wall.

> **You should see**
>
> All modules agree on the raw detection polarity and respond consistently on the chosen object.

If that does not happen: Do not treat a potentiometer position as a measured distance. Fix wiring, lighting or calibration before enabling motors.

STEP 3

Set motor inversion and active level

Open the obstacle sketch and copy your recorded motor direction settings. Set DETECTED_LEVEL for actual obstacle detection. The initial SPEED is 145; timed turns are estimates, not measured angles.

> **You should see**
>
> Code settings match your notes.

If that does not happen: Opposite sensor polarities require a teacher-reviewed code change; a single global setting cannot represent contradictory modules.

STEP 4

Upload, then prepare the run

Upload over isolated USB with motor battery OFF. The program starts its autonomous session at reset/power-up and stops after 60 seconds. For battery-only tests, unplug USB and restore approved external power only while everything is OFF. Lift the wheels before the first powered motor check.

> **You should see**
>
> A successful upload and mode-identifying serial message before the motor run.

If that does not happen: Check the selected file and core version. Do not enable motor supply just to see the message.

Check every decision

Front	Left	Right	Expected action
Clear	Any	Any	Forward
Blocked	Clear	Clear	Stop, turn left
Blocked	Clear	Blocked	Stop, turn left
Blocked	Blocked	Clear	Stop, turn right
Blocked	Blocked	Blocked	Stop, reverse, timed left U-turn

STEP 5

Test with wheels raised first

Present objects to create each row, keeping hands clear of wheels. After a turn completes the program stops briefly and re-reads sensors. With a clear front it then moves forward again. Teacher controls the switch throughout.

> **You should see**
>
> The observed wheel directions match the table and the session stops after about one minute from startup.

If that does not happen: If a turn is wrong, separate motor direction from sensor input errors using the earlier tests.

STEP 6

Run on a clear floor

Use soft, stable test objects and clear the rear area. Begin with one front obstacle. Then test a clear-side choice. Test all-blocked reverse only with the teacher guarding the robot. Switch OFF before repositioning.

> **You should see**
>
> A low-speed approach, stop, side choice and fresh sensor decision after the timed motion.

If that does not happen: Switch off if an object is missed, the chassis gets stuck, or the ESP32 resets. Recheck reflectivity, sensitivity and supply.

Adjust timed motions

Constant	Initial value	Purpose
REVERSE_MS	350 ms	Short reverse interval when all sides are blocked
TURN_MS	400 ms	Approximate side turn time
UTURN_MS	800 ms	Approximate U-turn time
SESSION_MS	60000 ms	Autonomous session limit

A flat floor, battery voltage and left/right motor differences change the resulting angle. Tune one value at a time and log the result. The program does not guarantee exactly 90° or 180°.

Finished when

- [ ] All three sensors calibrated for the actual obstacle.
- [ ] Each table row checked with wheels lifted.
- [ ] Reverse area clear and teacher supervised.
- [ ] One short floor run and a safe shutdown completed.

CHAPTER 12

Shut down & change modes

End every run with power off and a short test note.

BEFORE YOU START

The robot in a clear area and the switch reachable.

FINISH WHEN

The robot is powered down and ready for safe storage or reconfiguration.

End a run

STEP 1

Stop motion before touching the robot

For remote mode send S, then switch the motor battery OFF. For autonomous modes, switch OFF before lifting or repositioning; do not wait for a timed session to end when something is going wrong.

> **You should see**
>
> Wheels stopped and motor supply switched OFF.

If that does not happen: Use the reachable power switch if a serial command cannot be sent. Keep fingers away from moving wheels.

STEP 2

Disconnect and inspect

Close the serial terminal, unplug USB if connected, and let the teacher inspect unusual heat or damaged wiring. An adult handles cell removal, storage and charging arrangements. Do not leave a kit powered unattended.

> **You should see**
>
> No supply remains connected while you move wires or mounts.

If that does not happen: Stop and report unusual heat, smell, cell damage or a repeated reset. Do not resume testing until checked.

STEP 3

Record one useful observation

Write the mode, settings, expected result and what actually happened. Distinguish “compiled/uploaded” from “tested on the floor”. Record a failure clearly; it is useful evidence.

> **You should see**
>
> A short note that another student or your teacher can reproduce.

If that does not happen: Change one setting next time, rather than making several unrecorded edits.

Test log to print

Date / mode	Setting changed	Expected result	Observed result	Teacher note
_____	_____	_____	_____	_____
_____	_____	_____	_____	_____
_____	_____	_____	_____	_____

Change to another mode

1. Switch OFF and unplug USB before moving sensors.
2. Move the mounts to match the selected mode and check its OUT pin table.
3. Recalibrate for the new surface/object using motor-OFF diagnostics.
4. Copy known motor inversion and the appropriate sensor active level into the new mode sketch.
5. Upload using isolated USB. Recheck lifted-wheel direction/stop behavior before the next floor run.

The board runs only the last sketch uploaded. Uploading another mode replaces the previous program; nothing in this website changes the robot's mode.

CHAPTER 13

Find & fix a problem

Choose the symptom you can observe, then change one thing at a time.

BEFORE YOU START

Your exact error, sketch name, board/core selection and last successful step.

FINISH WHEN

You have an evidence-based next action or a clear report for your teacher.

Start with the symptom

For wiring changes: motor switch OFF and USB unplugged. For software-only checks: use the approved USB-isolated arrangement. Never keep retrying a jammed motor or a board that repeatedly browns out.

No COM port appears

Try a known data cable and another USB socket. Check Device Manager for CP210x. Install the official driver only if needed.

A powered LED does not prove USB data works.

Port busy / access denied

Close Serial Monitor, PuTTY and other serial applications, then reconnect/reselect the port. Ask lab staff about permissions.

Only one terminal should own the port.

ledcAttach / ledcWrite compiler error

Use esp32 by Espressif Systems core 3.x, original ESP32 target and the current sketch.

Core 2.x uses a different PWM API.

Upload cannot connect

Confirm USB port and board. Press/release EN to reset. If the exact board needs it, hold BOOT while upload begins connecting, then release when transfer starts.

Keep motor battery OFF; do not randomly change wiring.

Text is garbled

Set Serial Monitor to 115200. Reset once to catch the startup message.

Baud rate must match Serial.begin.

PING gives no PONG

Use 00_Serial_Test, uppercase PING, 115200 baud and Newline. Enter the text in Serial Monitor, not the editor.

PING is not a command in the remote sketch.

Only a short movement after F

This is the 800 ms deadman. Repeat valid movement commands for sustained movement; S stops.

Do not disable the timeout.

Motors never move

Switch OFF. Check ENA/ENB jumpers, IN pins, motor output pairs, ground, motor supply and wheel friction.

USB powering ESP32 does not automatically power motors.

One motor turns backwards

With power off, change the matching INVERT_LEFT or INVERT_RIGHT value, upload and retest.

Copy that setting into all three mode sketches.

ESP32 resets when motors start

Stop. Teacher measures regulator supply under motor load, checks wiring and USB/external supply isolation.

The assumed onboard 5 V path may be unstable; no unverified power fix is prescribed.

IR readings never change / flicker

Use the motor-OFF diagnostic. Check module labels, 3.3 V compatibility, OUT wiring, sensitivity, surface, height and light.

A disconnected input can float. Modules may have opposite polarity.

Line robot searches on a straight

Both-white is ambiguous in the flanking layout. The conservative timer can search even when centered.

This is a documented algorithm limitation, not proof of a faulty sensor.

Line robot stops and will not restart

Both-black or expired search latches stopped. Switch OFF, inspect/reposition and restart after checks.

The autonomous session also ends at 60 seconds.

Avoidance misses an obstacle

Try a matte object and recalibrate each module. Guard rear space during timed reverse.

Dark/transparent/shiny objects may not be reliably detected.

Bluetooth pairs but does not drive

Find the correct outgoing Classic SPP COM, close competing terminals, send S and look for actual STOP feedback.

Pairing alone is not serial communication. USB is the fallback.

Cannot open the website

Ask the host/teacher to start PHP, then refresh the provided local URL. No account is required.

Hardware commands go through a serial terminal, not this website.

Tell the teacher these five facts

- [ ] Mode / sketch name: _____.
- [ ] Last step that worked: _____.
- [ ] Exact error or observed action: _____.
- [ ] Board, USB/Bluetooth COM and core version: _____.
- [ ] What changed immediately before the problem: _____.

> **Do not guess the power fix**
>
> Do not add a random supply, charger or extra battery part to get past a reset. The power arrangement in this project is an unverified kit assumption requiring teacher measurement.

CHAPTER 14

Understand the parts & code

A short reference to read whenever a term is unfamiliar.

BEFORE YOU START

No practical test required. Use this alongside the setup steps.

FINISH WHEN

You can identify the terms and constants used in your sketch.

Plain-language glossary

Term	What it means here
ESP32	The microcontroller that runs one uploaded program at a time.
GPIO	A numbered signal pin. GPIO number is not the physical row position.
GND / common ground	Shared voltage reference between board, driver and sensors.
H-bridge / L298N	The driver switches motor current and direction; ESP32 pins only send control signals.
PWM / duty	Fast power switching. SPEED=145 is duty out of 255, not motor RPM or a measured voltage.
Digital IR sensor	A reflected-light threshold output, HIGH or LOW. It does not report exact distance.
Active level	The measured HIGH/LOW value that the program interprets as black or an obstacle.
USB COM / Bluetooth COM	Different Windows serial connections. The wireless port requires Classic SPP support.

Term	What it means here
Verify / Upload	Compile source / send the compiled program to the board.
setup() / loop()	Run initialization once / repeat decision logic.
millis() / delay()	Measure elapsed time / pause processing. The remote timeout uses millis().
Deadman	Stop if movement commands are not refreshed within the allowed time.

Read the motor function

`drive(left,right)` takes signed PWM values. Positive means forward after your inversion calibration, negative means reverse, and zero means stop. It calls `oneMotor()` for each side, clears PWM before changing direction and then applies the new duty.

Where to edit safely

```
const int SPEED=145;
const bool INVERT_LEFT=false, INVERT_RIGHT=false;
const int DETECTED_LEVEL=LOW; // only if your measurements confirm it
```

Edit one setting at a time near the top of the relevant sketch. Save, Verify and Upload after each change. Every robot .ino is self-contained; changing one does not automatically update the others. Do not change the GPIO map in just one place without updating the wiring and documentation.

Verified and unverified

The original three robot sketches compiled for `esp32:esp32:esp32` using core 3.3.11. Website software checks do not verify the actual electrical circuit, sensor model, mounting fit, Bluetooth COM compatibility or floor behavior. Final kit photos, fastener specifications, board/module confirmation and support contact are still pending.

Official resources

- Arduino IDE 2 documentation
- Espressif Arduino-ESP32 installation
- Espressif PWM API
- Espressif Bluetooth Classic examples
- Silicon Labs CP210x drivers